

MongoDB Complete Notes (Basic → Advanced)

এই ডকুমেন্টে তুমি **MongoDB পুরোটা শিখবে** — একদম zero থেকে advanced level পর্যন্ত। প্রতিটা জিনিসে থাকবে: - ♦
এটা কী - ♦ কেন ব্যবহার করি - ♦ কিভাবে কাজ করে - ♦ Real example

এই নোট শেষ করলে তুমি confidently বলতে পারবে: "আমি MongoDB বুঝি" ✅

1 MongoDB কী?

MongoDB হলো একটি **NoSQL, Document-based Database**।

কেন MongoDB?

- SQL এর মতো table নাই
- Data থাকে **JSON-like document (BSON)** আকারে
- Flexible structure
- High performance
- Big data friendly

🔗 Modern web & mobile app-এর জন্য perfect

2 SQL vs NoSQL (MongoDB)

SQL	MongoDB
Table	Collection
Row	Document
Column	Field
Fixed Schema	Dynamic Schema
JOIN heavy	Embed / Reference

3 Database → Collection → Document

```
Database
├─ Collection (users)
│   └─ Document
│       {
```

```
"name": "Fatima",  
"age": 25  
}
```

4 Install MongoDB & Mongo Shell

- MongoDB Server
- MongoDB Compass (GUI)
- mongosh (CLI)

```
mongosh
```

5 Basic Commands

```
show dbs  
use testDB  
show collections  
db.users.find()
```

6 Insert Data

insertOne

```
db.users.insertOne({ name: "Fatima", age: 25 })
```

insertMany

```
db.users.insertMany([  
  { name: "A", age: 20 },  
  { name: "B", age: 22 }  
)
```

7 Read / Find

```
db.users.find({ age: 25 })
db.users.findOne({ name: "Fatima" })
```

Comparison Operator

```
{ age: { $gt: 18 } }
```

8 Projection (Select field)

```
db.users.find({}, { name: 1, _id: 0 })
```

9 Update

updateOne

```
db.users.updateOne(
  { name: "Fatima" },
  { $set: { age: 26 } }
)
```

updateMany

```
db.users.updateMany(
  { age: { $lt: 18 } },
  { $set: { isMinor: true } }
)
```

10 Delete

```
db.users.deleteOne({ name: "A" })
db.users.deleteMany({ age: { $lt: 18 } })
```

1 1 Operators

Logical

- \$and
- \$or
- \$not

Array

- \$in
- \$all
- \$size

1 2 Embedded Document

```
{
  name: "Fatima",
  address: {
    city: "Dhaka",
    zip: 1200
  }
}
```

কেন ব্যবহার করি?

🔗 Fast read

1 3 Array of Documents

```
{
  name: "Fatima",
  skills: ["JS", "React", "Node"]
}
```

1 4 Index

```
db.users.createIndex({ email: 1 })
```

কেন?

🔗 Query fast করার জন্য

1 5 Unique Index

```
db.users.createIndex({ email: 1 }, { unique: true })
```

1 6 Sorting, Limit, Skip

```
db.users.find().sort({ age: -1 }).limit(5).skip(5)
```

1 7 Aggregation Pipeline (Core Concept)

```
db.users.aggregate([
  { $match: { age: { $gte: 18 } } },
  { $group: { _id: "$city", total: { $sum: 1 } } }
])
```

Stage মানে কী?

🔗 Step by step data processing

1 8 Lookup (Join)

```
{
  $lookup: {
    from: "orders",
    localField: "_id",
    foreignField: "userId",
    as: "orders"
  }
}
```

1 9 Transactions

```
session.startTransaction()
```

কেন?

🏦 Banking / Order system

2 0 Data Modeling Strategy

Embed vs Reference

Embed	Reference
Fast read	Flexible
Less join	Reusable

2 1 Performance Tips

- Index use করো
 - Projection use করো
 - Large document avoid করো
 - Aggregation optimize করো
-

2 2 MongoDB in Real Project

- User Collection
 - Order Collection
 - Payment Collection
 - Logs Collection
-

2 3 Advanced Aggregation (Deep Dive)

Common Stages Explained

\$match

```
{ $match: { status: "active" } }
```

🔗 SQL WHERE এর মতো

\$project

```
{ $project: { name: 1, email: 1, _id: 0 } }
```

🔗 কোন field দেখাবো control করা

\$group

```
{
  $group: {
    _id: "$role",
    totalUser: { $sum: 1 },
    avgAge: { $avg: "$age" }
  }
}
```

🔗 Report / Analytics এর জন্য

\$sort

```
{ $sort: { createdAt: -1 } }
```

\$limit & \$skip

```
{ $skip: 10 },
{ $limit: 5 }
```

2 4 Facet (Multiple Pipeline Result)

```
db.users.aggregate([
  {
    $facet: {
      activeUsers: [{ $match: { active: true } }],
      inactiveUsers: [{ $match: { active: false } } ]
    }
  }
])
```

```
}  
])
```

🔗 এক query তে multiple result

2.5 Unwind (Array Break)

```
{ $unwind: "$skills" }
```

🔗 Array কে line-by-line document বানায়

2.6 Text Search

```
db.posts.createIndex({ title: "text", body: "text" })  
  
db.posts.find({ $text: { $search: "mongodb" } })
```

2.7 TTL Index (Auto Delete)

```
db.sessions.createIndex(  
  { createdAt: 1 },  
  { expireAfterSeconds: 3600 }  
)
```

🔗 OTP, Session auto delete

2.8 Atomic Operations

```
db.accounts.updateOne(  
  { _id: userId, balance: { $gte: 500 } },  
  { $inc: { balance: -500 } }  
)
```

🔗 Race condition avoid

2 9 Schema Validation (Mongo Level)

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email"],
      properties: {
        email: { bsonType: "string" }
      }
    }
  }
})
```

🔒 DB level protection

3 0 Sharding (High Scale)

- Data multiple server এ ভাগ করা
- Huge traffic handle

🔒 Enterprise level feature

3 1 Backup & Restore

```
mongodump
mongorestore
```

3 2 MongoDB Security

- Auth enable
 - Role based access
 - IP whitelist
 - TLS/SSL
-

3 3 Real SaaS Database Design Example

User Collection

```
{
  _id,
  name,
  email,
  role,
  createdAt
}
```

Order Collection

```
{
  userId,
  items: [],
  total,
  status,
  createdAt
}
```

3 4 Common Mistakes

- Over populate
- No index
- Huge document
- No validation

Final Outcome (Advanced)

এই MongoDB advanced নোট শেষ করলে তুমি: - Production-grade query লিখতে পারবে - Analytics & report বানাতে পারবে - Large scale SaaS DB design করতে পারবে - Interview confidently দিতে পারবে

 Next Suggestion: - MongoDB + Mongoose Combined Architecture - Advanced Aggregation Project - SaaS-ready Database Blueprint